

Diseño e Implementación de un Lenguaje

Fecha: 3 de septiembre de 2026

Nombre del lenguaje: B#

1. Equipo

Nombre	Apellido	Legajo	E-mail
Milagros Maria	Pipet	65104	mpipet@itba.edu.ar
Martina	Malleville	63335	mmalleville@itba.edu.ar
Carolina Luciana	Laurenza	64647	claurenza@itba.edu.ar
Victoria Helena	Park	64498	vpark@itba.edu.ar

2. Repositorio

La solución será versionada en:

<https://github.com/vpark2003/TP-TLA.git>

Nombre del lenguaje: B#

3. Dominio

3.1. Descripción

El dominio seleccionado es la **composición y estructuración de música**, particularmente la descripción de canciones mediante sus componentes musicales fundamentales:

- Melodías
- Acordes
- Ritmos
- Instrumentos
- Secciones

La salida será en formato **MIDI**.

El objetivo de B# es permitir que un usuario pueda describir una canción utilizando una sintaxis cercana al lenguaje utilizado por un músico o compositor, sin tener que ocuparse de los detalles técnicos necesarios para representar y reproducir la información musical.

Una canción será modelada como una composición formada por diferentes secciones, como:

- Versos
- Estribillos

- Puentes

Cada sección podrá contener diferentes elementos musicales, tales como:

- Notas
- Silencios
- Acordes
- Patrones rítmicos

El lenguaje permitirá además:

- Reutilizar secciones y patrones.
- Definir diferentes instrumentos.
- Controlar parámetros generales de la composición, como el tempo y el compás.

La intención es que el usuario pueda expresar **qué música desea construir** mediante conceptos propios del dominio, mientras que el compilador se encargará de transformar dicha descripción en una representación musical procesable.

Este enfoque busca que el lenguaje sea un **Domain Specific Language (DSL)**, ya que su sintaxis estará diseñada específicamente alrededor de los conceptos musicales que se desean manipular, en lugar de utilizar construcciones genéricas de un lenguaje de propósito general.

3.2. Motivación

La representación de una composición musical mediante lenguajes tradicionales requiere trabajar con estructuras y detalles que no necesariamente pertenecen al modelo mental de un músico.

Por ejemplo, representar una melodía puede implicar manipular estructuras de datos, índices, números asociados a notas o valores temporales.

B# busca abstraer estos detalles y permitir una descripción más cercana al dominio:

```
C4 quarter  
E4 quarter  
G4 half
```

De esta forma, una persona puede identificar directamente la información musical representada.

El lenguaje también busca abstraer la estructura de una canción. En lugar de repetir manualmente una misma secuencia de notas, el usuario podrá definir una sección una única vez y reutilizarla:

```
repeat chorus 2
```

Esto permite aprovechar mecanismos propios de los lenguajes de programación para representar conceptos musicales de mayor nivel.

3.3. Especificidad del dominio

El dominio es específico, ya que se encuentra concentrado en la composición y estructuración de piezas musicales.

Las entidades principales del dominio presentan relaciones estrechas entre sí:

- Una canción está formada por secciones.
- Una sección contiene eventos musicales.
- Los eventos pueden ser notas, silencios, acordes o patrones.
- Las notas poseen altura y duración.
- Las notas podrán tener alteraciones como sostenido (#) y bemol (b)
- Los acordes representan estructuras armónicas.
- Los instrumentos determinan la interpretación de los eventos.
- El tempo y el compás determinan características temporales de la composición.

Por lo tanto, el lenguaje podrá incorporar estas entidades directamente como abstracciones del modelo computacional.

4. Construcciones, prestaciones y funcionalidades

B# deberá permitir las siguientes construcciones.

4.1. Definición de una canción

Se podrá crear una canción indicando su nombre:

```
song "Midnight"
```

Una canción podrá contener su configuración general y las diferentes secciones que la componen.

4.2. Configuración del tempo

Se podrá establecer el tempo de una canción mediante BPM:

```
tempo 120
```

El valor deberá ser un número entero positivo.

4.3. Definición del compás

Se podrá indicar el compás de la composición:

```
time 4/4
```

El compás estará formado por numerador y denominador enteros positivos. El denominador deberá corresponder a una potencia de dos, como 2, 4, 8 o 16.

4.4. Instrumentos

El usuario podrá seleccionar instrumentos para interpretar diferentes partes de una canción:

```
instrument piano
```

Se contemplará un conjunto predefinido de instrumentos.

Entre ellos podrán encontrarse:

- piano
- guitar
- bass
- violin
- drums

El conjunto definitivo podrá ampliarse durante la implementación.

4.5. Notas musicales

Se podrán representar notas mediante su **nombre**, **octava** y **duración**:

```
C4 quarter
E4 eighth
G4 half
```

Las notas disponibles serán:

```
A B C D E F G
```

(LA, SI, DO, RE, MI, FA, SOL)

También podrán incluir alteraciones:

```
C#4
Bb3
```

La octava será un valor entero entre 0 y 8 inclusive.

4.6. Duraciones

Las notas y otros eventos musicales podrán utilizar diferentes duraciones:

Duración	Equivalencia	Tiempos
whole	Redonda	4
half	Blanca	2
quarter	Negra	1
eighth	Corchea	½
sixteenth	Semicorchea	¼

También se podrá representar un silencio:

```
rest quarter
```

4.7. Acordes

El lenguaje permitirá representar acordes utilizando su raíz y tipo:

```
chord C  
chord Am
```

Los acordes podrán utilizarse dentro de una sección para representar la armonía de la composición.

4.8. Progresiones armónicas

Se podrá definir una secuencia de acordes:

```
progression pop {  
  C  
  Am  
  F  
  G  
}
```

Una progresión podrá ser utilizada posteriormente dentro de una sección.

```
use pop
```

4.9. Secciones musicales

La estructura de una canción podrá dividirse en secciones identificables:

```
verse {  
  C4 quarter  
  E4 quarter  
  G4 half  
}  
  
chorus {  
  A4 quarter  
  G4 quarter  
  E4 half  
}
```

Se contemplarán, como mínimo, los siguientes conceptos:

- `intro`
- `verse`
- `chorus`

- `bridge`
- `outro`

Las secciones podrán ser referenciadas por su nombre.

4.10. Reutilización y repetición

Una sección existente podrá reutilizarse sin necesidad de redefinir su contenido:

```
repeat chorus 2
```

También se permitirá utilizar una sección directamente:

```
verse  
chorus  
verse  
chorus
```

Esto permitirá representar estructuras musicales repetitivas de manera compacta.

La construcción `repeat <section> <n>` permite reutilizar una sección previamente definida.

Por otro lado, la estructura `repeat <n> { ... }` permite repetir un conjunto de instrucciones o eventos musicales una cantidad determinada de veces.

4.11. Tracks o capas musicales

Una canción podrá contener diferentes capas musicales independientes.

Por ejemplo:

```
track melody using piano {  
    C4 quarter  
    E4 quarter  
    G4 half  
}  
  
track bass using bass {  
    C2 half  
    G2 half  
}
```

Cada `track` podrá asociarse a un instrumento.

4.12. Patrones musicales

Se podrán definir patrones reutilizables:

```
pattern riff {  
    C4 eighth  
    E4 eighth  
    G4 quarter  
}
```

Posteriormente podrán utilizarse dentro de diferentes secciones:

```
use riff
```

Un `pattern` representa una secuencia reutilizable de eventos musicales. Los patrones deberán ser definidos antes de ser utilizados.

Esto permitirá evitar la repetición de código y favorecer la reutilización de estructuras musicales.

4.13. Transposición

Se podrá modificar la altura de una melodía o patrón mediante una operación de transposición:

```
pattern riff {  
    C4 eighth  
    E4 eighth  
    G4 quarter  
}  
  
transpose riff +2
```

La construcción `transpose` recibe el nombre de un patrón o sección previamente definida y una cantidad entera de semitonos. La operación genera una versión transpuesta del elemento indicado.

4.14. Control de volumen

Los tracks podrán definir un nivel de volumen:

```
track melody using piano volume 80 {  
    C4 quarter  
    E4 quarter  
    G4 half  
}
```

El volumen será un entero entre 0 y 127 inclusive.

4.15. Variables

El lenguaje podrá utilizar variables para almacenar valores simples y conceptos propios del dominio musical.

Se contemplarán, como mínimo:

- `integer`
- `string`
- `boolean`
- `note`
- `duration`
- `chord`

Por ejemplo:

```
tempoValue = 120
```

4.16. Operadores

Se contemplarán los siguientes operadores:

Operadores aritméticos

```
+ - * /
```

Operadores relacionales

```
< > = != <= >=
```

Operadores lógicos

```
AND OR NOT
```

Estos operadores podrán utilizarse en expresiones y condiciones.

4.17. Estructuras de control

El lenguaje podrá incluir estructuras de control para permitir composiciones más expresivas.

Repetición

```
repeat 4 {  
    C4 quarter  
    E4 quarter  
}
```

4.18. Generación del artefacto musical

El compilador recibirá como entrada un programa escrito en B#.

Como resultado de la compilación se generará una representación musical, principalmente un archivo **MIDI**, que contendrá la composición descrita por el usuario.

Flujo esperado

```
Programa B#  
↓  
Análisis léxico  
↓  
Análisis sintáctico  
↓  
AST
```



```
↓  
Análisis semántico  
↓  
Generación de código  
↓  
Archivo MIDI
```

El mecanismo concreto de generación del archivo MIDI será definido durante las etapas posteriores del proyecto.

5. Sistema de tipos

B# utilizará un sistema de tipos que combine tipos básicos con tipos específicos del dominio musical.

5.1. Tipos básicos

Integer

Representa números enteros.

```
tempoValue = 120
```

String

Representa cadenas de caracteres.

```
songName = "Midnight"
```

Boolean

Representa valores lógicos:

```
true  
false
```

5.2. Tipos específicos del dominio

Note

Representa una nota musical.

```
C4
```

Duration

Representa una duración musical.

```
quarter
```

Chord

Representa un acorde.

```
Am
```

Section

Representa una sección de una canción.

```
chorus
```

Pattern

Representa una secuencia musical reutilizable.

```
riff
```

Track

Representa una capa independiente de la composición.

5.3. Tipado

Se propone utilizar tipado estático.

De esta manera, las operaciones incompatibles podrán ser detectadas durante la compilación antes de generar el artefacto final.

Por ejemplo:

```
C4 + "hello"
```

deberá ser rechazada debido a que los operandos pertenecen a tipos incompatibles.

Esto permite detectar errores relacionados con el dominio antes de generar el archivo musical.

6. Casos de prueba

Los siguientes casos se plantean como **Unit Tests**, buscando que cada uno sea compacto y específico para probar una funcionalidad determinada.

6.1. Casos de aceptación

A1 - Canción mínima

```
song "Test"
```

Resultado esperado: aceptación.

Verifica que pueda declararse una canción.

A2 - Definición de tempo

```
tempo 120
```

Resultado esperado: aceptación.

Verifica la declaración de un tempo válido.

A3 - Nota

```
C4 quarter
```

Resultado esperado: aceptación.

Verifica la representación de una nota válida.

A4 - Silencio

```
rest quarter
```

Resultado esperado: aceptación.

Verifica la representación de un silencio.

A5 - Acorde

```
chord C major
```

Resultado esperado: aceptación.

Verifica la representación de un acorde válido.

A6 - Sección

```
verse {  
    C4 quarter  
}
```

Resultado esperado: aceptación.

Verifica la definición de una sección.

A7 - Repetición

```
repeat chorus 2
```

Resultado esperado: aceptación cuando `chorus` haya sido previamente definida.

Verifica la reutilización de una sección.

A8 - Instrumento

```
instrument piano
```

Resultado esperado: aceptación.

Verifica la selección de un instrumento válido.

A9 - Patrón reutilizable

```
pattern riff {  
    C4 eighth  
}
```

Resultado esperado: aceptación.

Verifica la definición de un patrón musical.

A10 - Track

```
track melody using piano {  
    C4 quarter  
}
```

Resultado esperado: aceptación.

Verifica la creación de una capa musical asociada a un instrumento.

A11 - Transposición

```
transpose melody +2
```

Resultado esperado: aceptación cuando `melody` exista.

Verifica la aplicación de una operación específica del dominio.

A12 - Canción estructurada

```
song "Example" {  
    verse {  
        C4 quarter  
    }  
}
```

```
chorus {  
    G4 quarter  
}  
  
repeat chorus 2  
}
```

Resultado esperado: aceptación.

Verifica la combinación de canción, secciones y repetición.

6.2. Casos de rechazo

R1 - Nota inválida

```
H4 quarter
```

Resultado esperado: rechazo.

La nota **H** no pertenece al conjunto de notas permitido.

R2 - Duración inválida

```
C4 banana
```

Resultado esperado: rechazo.

banana no representa una duración musical válida.

R3 - Tempo inválido

```
tempo -120
```

Resultado esperado: rechazo.

El tempo debe ser un valor positivo.

R4 - Sección inexistente

```
repeat chorus 2
```

sin haber definido previamente **chorus**.

Resultado esperado: rechazo.

La referencia a una sección inexistente constituye un error semántico.

R5 - Instrumento inexistente

```
instrument spaceship
```

Resultado esperado: rechazo.

`spaceship` no pertenece al conjunto de instrumentos soportados.

R6 - Tipo incompatible

```
tempoValue = C4 + "hello"
```

Resultado esperado: rechazo.

La expresión intenta operar valores de tipos incompatibles.

R7 - Octava inválida

```
C99 quarter
```

Resultado esperado: rechazo.

La octava se encuentra fuera del rango soportado por el lenguaje.

R8 - Repetición inválida

```
repeat chorus 0
```

Resultado esperado: rechazo.

La cantidad de repeticiones debe ser positiva.

7. Ejemplos

7.1. Ejemplo 1 - Canción simple

El siguiente programa representa una canción pequeña con un verso y un estribillo:

```
song "Midnight" {  
  tempo 120  
  time 4/4  
  instrument piano  
  
  verse {  
    C4 quarter  
    E4 quarter  
    G4 half  
  }  
  
  chorus {  
    A4 quarter  
    G4 quarter  
    E4 half  
  }  
}
```

```
}  
}
```

¿Qué representa?

El programa define una canción llamada **Midnight**, establece un tempo de **120 BPM** y un compás de **4/4**.

Luego define dos secciones:

- `verse`
- `chorus`

Cada sección contiene una secuencia de notas con diferentes duraciones.

El compilador deberá interpretar estas construcciones y producir la representación musical correspondiente.

7.2. Ejemplo 2 - Canción con reutilización y múltiples capas

El siguiente ejemplo utiliza diferentes tracks y reutiliza una sección:

```
song "Summer Lights" {  
  tempo 128  
  time 4/4  
  
  track melody using piano {  
    C4 quarter  
    E4 quarter  
    G4 half  
  
    A4 quarter  
    G4 quarter  
    E4 half  
  }  
  
  track bass using bass {  
    C2 half  
    G2 half  
  }  
}
```

En este caso, la composición posee **dos capas independientes**:

- Una melodía interpretada por piano.
- Una línea de bajo.

La melodía está organizada en las secciones `verse` y `chorus`, mientras que la sección `chorus` es reutilizada mediante una construcción de repetición.

Este ejemplo muestra cómo B# permite representar no solamente notas individuales, sino también la estructura y organización de una composición musical.

8. Resumen de la propuesta

B# será un **DSL orientado a la composición musical**, diseñado para permitir la descripción de canciones mediante conceptos propios del dominio.

Principales abstracciones

- Canciones
- Secciones
- Notas
- Silencios
- Acordes
- Progresiones
- Instrumentos
- Tracks
- Patrones
- Tempo
- Compás
- Transposición
- Repetición

El lenguaje buscará que la representación de una composición sea cercana al modelo mental de un músico, ocultando los detalles técnicos de la representación final.

El artefacto principal de salida será un archivo MIDI generado a partir del programa B#.

La sintaxis presentada en este documento constituye la propuesta inicial para la implementación del frontend y podrá recibir ajustes durante el desarrollo, procurando mantener sus conceptos fundamentales.